The Complete Machine Learning Algorithms Guide

Created by	https://www.instagram.com/datasciencebrain

Every Algorithm You Need to Know — From Basics to Advanced

1. Linear Regression

What Is It?

Linear Regression is the simplest and most foundational supervised learning algorithm. It models the relationship between input features and a continuous output variable by fitting a straight line (or hyperplane in higher dimensions) through the data.

Key Formula

`Y = b0 + b1X1 + b2X2 + ... + bnXn`

- `Y` = predicted output
- `b0` = bias/intercept
- `b1...bn` = learned weights
- `X1...Xn` = input features

Core Assumptions

- **Linearity** — relationship between X and Y is linear
- **Independence** — observations are independent of each other
- **Homoscedasticity** — constant variance of errors
- **No multicollinearity** — features are not highly correlated

How It Learns

It minimizes the **Mean Squared Error (MSE)** between predicted and actual values using **Gradient Descent** or the **Normal Equation**.

`MSE = (1/n) Σ(Yi - Ŷi)²`

Pros

- Simple and highly interpretable
- Fast to train and predict
- Works well when relationship is truly linear
- Great baseline model

Cons

- Sensitive to outliers
- Assumes linearity — fails with complex patterns
- Assumes no multicollinearity
- Cannot model non-linear relationships without feature engineering

When Not to Use

- When data has strong non-linear relationships
- When features are highly correlated (multicollinearity)
- When there are many irrelevant features

Real-World Use Cases

- House price prediction
- Sales forecasting
- Stock price estimation
- Risk assessment in finance

Interview Tips

> **Q: What is the difference between Linear and Logistic Regression?**
> Linear regression predicts continuous values. Logistic regression predicts

> probabilities for classification problems using a sigmoid function.

> **Q: How do you handle outliers in Linear Regression?**
> Use robust regression, remove outliers, or switch to MAE loss instead of MSE.

---

2. Logistic Regression

What Is It?

Despite the name, Logistic Regression is a **classification** algorithm. It predicts the probability of a binary outcome using the sigmoid (logistic) function, which squashes any input into a value between 0 and 1.

Key Formula

`P = 1 / (1 + e^-(b0 + b1X1 + ... + bnXn))`

- Output is always between 0 and 1
- Threshold (usually 0.5) converts probability to class label

Core Assumptions

- **Log-odds linearity** — linear relationship between features and log-odds
- **Binary outcome** — target variable is 0 or 1
- **No multicollinearity** — features not highly correlated
- **Large sample size** — works better with more data

How It Learns

Minimizes **Binary Cross-Entropy Loss** using Gradient Descent:

`Loss = -[y·log(p) + (1-y)·log(1-p)]`

Pros

- Probabilistic output (gives confidence scores)
- Highly interpretable — coefficients show feature importance
- Works well with linearly separable data
- Fast training and prediction

Cons

- Weak with non-linear decision boundaries
- Requires feature scaling
- Sensitive to outliers
- Struggles with many irrelevant features

When Not to Use

- When data is highly non-linear
- When classes are heavily imbalanced (without adjustments)
- When you need to model complex feature interactions

Real-World Use Cases

- Spam detection
- Disease diagnosis (cancer/no cancer)
- Credit card fraud detection
- Customer churn prediction

Interview Tips

> **Q: Why is it called Logistic Regression if it is used for classification?**
> It uses regression internally to model log-odds, but the output is used for classification via a threshold.

> **Q: What is the difference between sigmoid and softmax?**
> Sigmoid is for binary classification (2 classes). Softmax is for multi-class classification (3+ classes).

---

3. Decision Tree

What Is It?

A Decision Tree is a tree-structured model that splits data into subsets based on feature values. Each internal node represents a decision rule, each branch

represents an outcome, and each leaf represents a final prediction. Works for both classification and regression.

Key Logic

```
Recursive Binary Splitting:
- At each node, find the feature and threshold that best splits the data
- Measure quality using Gini Impurity or Information Gain (Entropy)
- Repeat until stopping criteria is met
```

Splitting Criteria

- **Gini Impurity**: `G = 1 - Σ(pi²)`
- **Entropy**: `E = -Σ(pi · log2(pi))`
- **Information Gain**: `IG = Entropy(parent) - weighted avg Entropy(children)`

Core Assumptions

- No strict statistical assumptions
- Works with both numerical and categorical data
- Can handle non-linear relationships

Pros

- Extremely easy to interpret and visualize
- No need for feature scaling
- Handles both numerical and categorical data
- Captures non-linear relationships

Cons

- Highly prone to overfitting (especially deep trees)
- Unstable — small data changes can drastically change the tree
- Biased toward features with more levels
- Not great for continuous target variables

When Not to Use

- When you need high accuracy (use ensemble methods instead)
- When data is noisy
- When dataset is very small

Hyperparameters to Tune

- `max_depth` — controls tree depth to prevent overfitting
- `min_samples_split` — minimum samples needed to split a node
- `min_samples_leaf` — minimum samples in a leaf node
- `max_features` — number of features to consider at each split

Real-World Use Cases

- Loan default prediction
- Medical diagnosis
- Customer segmentation
- Rule extraction for business logic

Interview Tips

> **Q: How do you prevent overfitting in a Decision Tree?**
> Pruning, limiting max_depth, increasing min_samples_split, or using ensemble methods like Random Forest.

---

4. Random Forest

What Is It?

Random Forest is an ensemble method that builds multiple Decision Trees during training and combines their predictions. It uses **Bagging (Bootstrap Aggregating)** — each tree is trained on a random subset of data and features, reducing variance and overfitting.

Key Logic

```
1. Draw n random bootstrap samples from training data
2. Train a Decision Tree on each sample
3. At each node, consider only a random subset of features
4. For classification: majority vote across all trees
5. For regression: average prediction across all trees
```

Core Assumptions

- Tree independence — each tree should be as different as possible
- Enough trees to stabilize predictions (typically 100–500)

Pros

- High accuracy and robustness
- Resistant to overfitting compared to single trees
- Handles missing values well
- Provides feature importance scores
- Works well out of the box with minimal tuning

Cons

- Slower than single Decision Trees
- Less interpretable (black box)
- High memory usage with many trees
- Can be slow for real-time prediction

When Not to Use

- When you need real-time low-latency predictions
- When model interpretability is critical
- When dataset is very small

Hyperparameters to Tune

- `n_estimators` — number of trees (more = better but slower)
- `max_depth` — depth of each tree
- `max_features` — features considered at each split (`sqrt` for classification)

- `min_samples_leaf` — minimum leaf size

Real-World Use Cases

- Fraud detection
- Medical diagnosis
- Stock market analysis
- Remote sensing / satellite image classification

Interview Tips

> **Q: What is the difference between Bagging and Boosting?**
> Bagging trains trees in parallel on random subsets, reducing variance. Boosting trains trees sequentially, each correcting the previous one's errors, reducing bias.

---

5. Gradient Boosting

What Is It?

Gradient Boosting is a powerful ensemble technique that builds trees **sequentially**, where each new tree corrects the errors (residuals) of the previous ensemble. Popular implementations include **XGBoost, LightGBM, and CatBoost**.

Key Logic

```
1. Start with a simple base model (e.g., mean prediction)
2. Calculate residuals (errors) of current model
3. Train a new tree to predict those residuals
4. Add new tree to ensemble with a learning rate
5. Repeat until n_estimators reached
```

Key Formula

```
F(x) = F0(x) + η·h1(x) + η·h2(x) + ... + η·hn(x)
```

- `F0` = initial prediction
- `η` = learning rate
- `h` = each new tree

Core Assumptions

- Sequential dependency — each tree depends on previous ones
- Works best with structured/tabular data

Pros

- State-of-the-art accuracy on tabular data
- Handles mixed data types well
- Built-in feature importance
- Flexible loss functions

Cons

- Prone to overfitting if not tuned properly
- Slower to train than Random Forest
- Many hyperparameters to tune
- Less interpretable

When Not to Use

- When interpretability is critical
- When training time is a constraint
- When dataset is very small

Hyperparameters to Tune

- `n_estimators` — number of boosting rounds
- `learning_rate` — shrinks contribution of each tree (lower = more robust)
- `max_depth` — depth of each tree (3–5 usually optimal)
- `subsample` — fraction of samples used per tree
- `colsample_bytree` — fraction of features used per tree

Real-World Use Cases

- Credit scoring
- Click-through rate prediction
- Kaggle competitions (most winning solutions)
- Anomaly detection

Interview Tips

> **Q: What is the difference between Random Forest and Gradient Boosting?**
> RF builds trees in parallel independently. GB builds trees sequentially, each fixing the previous model's errors. GB is usually more accurate but slower and more prone to overfitting.

---

6. Support Vector Machine (SVM)

What Is It?

SVM finds the optimal **hyperplane** that maximizes the margin between two classes. Points closest to the hyperplane are called **support vectors**. Using the **kernel trick**, it can handle non-linear boundaries by mapping data to higher dimensions.

Key Logic

```
Maximize: 2/||w|| (margin between classes)
Subject to: yi(w·xi + b) ≥ 1 for all points

Kernel Trick:
- Linear: K(x,y) = x·y
- RBF: K(x,y) = exp(-γ||x-y||²)
- Polynomial: K(x,y) = (x·y + c)^d
```

Core Assumptions

- Data should be scaled/normalized
- Works best when classes are separable (with some margin)

Pros

- Effective in high-dimensional spaces
- Memory efficient (only uses support vectors)

- Versatile with different kernel functions
- Works well with small to medium datasets

Cons

- Slow on large datasets ($O(n^2)$ to $O(n^3)$ complexity)
- Sensitive to feature scaling
- Hard to interpret
- Kernel and C parameter selection is tricky

When Not to Use

- Very large datasets
- When data is noisy with overlapping classes
- When you need probabilistic outputs natively

Key Hyperparameters

- `C` — regularization (high C = less margin, more overfitting)
- `kernel` — linear, rbf, poly, sigmoid
- `gamma` — influence of single training point (RBF kernel)

Real-World Use Cases

- Facial recognition
- Text categorization
- Image classification
- Bioinformatics (protein classification)

Interview Tips

> **Q: What is the kernel trick in SVM?**
> The kernel trick computes dot products in a high-dimensional space without explicitly transforming the data, making non-linear SVM computationally feasible.

---

7. K-Nearest Neighbors (KNN)

What Is It?

KNN is a simple, instance-based learning algorithm. For a new data point, it finds the K nearest neighbors in the training set and predicts based on majority vote (classification) or average (regression). It is a **lazy learner** — no training phase.

Key Logic

```
1. Choose value of K
2. Calculate distance to all training points
   - Euclidean: d = √Σ(xi - yi)²
   - Manhattan: d = Σ|xi - yi|
3. Select K nearest neighbors
4. Majority vote (classification) or mean (regression)
```

Core Assumptions

- Feature scaling is critical — all features must be on same scale
- Similar points exist close together in feature space

Pros

- Simple to understand and implement
- No training phase
- Naturally handles multi-class problems
- Adapts well to new training data

Cons

- Slow at prediction time (O(n) per query)
- Sensitive to irrelevant and redundant features
- Sensitive to noisy data
- High memory usage (stores all training data)

When Not to Use

- High-dimensional data (curse of dimensionality)

- Large datasets (too slow)
- When features have very different scales (without normalization)

Key Hyperparameters

- `K` — number of neighbors (odd K to avoid ties)
- `distance metric` — Euclidean, Manhattan, Minkowski
- `weights` — uniform or distance-based

Real-World Use Cases

- Recommender systems
- Anomaly detection
- Image recognition
- Medical diagnosis

Interview Tips

> **Q: How do you choose the optimal K in KNN?**
> Use cross-validation, plot error rate vs K, and choose K where error is minimized. Typically K = √n is a good starting point.

---

8. Naive Bayes

What Is It?

Naive Bayes is a probabilistic classifier based on **Bayes' Theorem** with the "naive" assumption that all features are **independent** of each other given the class label. Despite this strong assumption, it works surprisingly well for text classification.

Key Formula

`P(Class|Features) = P(Features|Class) × P(Class) / P(Features)`

```
Naive assumption:
P(X1,X2,...Xn|Class) = P(X1|Class) × P(X2|Class) × ... × P(Xn|Class)
```

Variants

- **Gaussian NB** — continuous features, assumes normal distribution
- **Multinomial NB** — count data, text classification
- **Bernoulli NB** — binary features, document classification

Core Assumptions

- Features are conditionally independent given the class
- Works best with large feature sets

Pros

- Extremely fast training and prediction
- Works well with small datasets
- Excellent for text classification
- Handles high-dimensional data well
- No need for feature scaling

Cons

- Feature independence assumption rarely holds in reality
- Poor probability estimates
- Fails when features are correlated
- Cannot learn feature interactions

When Not to Use

- When features are highly correlated
- When precise probability estimates are needed
- Complex real-world data with many feature dependencies

Real-World Use Cases

- Spam filtering
- Sentiment analysis

- Document classification
- Medical diagnosis

Interview Tips

> **Q: Why is Naive Bayes called "naive"?**
> Because it naively assumes all features are independent of each other, which is rarely true in practice but still yields good results.

---

9. K-Means

What Is It?

K-Means is an unsupervised clustering algorithm that partitions data into **K clusters** by minimizing the within-cluster variance (inertia). Each data point belongs to the cluster with the nearest centroid.

Key Logic

```
1. Initialize K centroids randomly
2. Assign each point to nearest centroid
3. Recalculate centroids as mean of assigned points
4. Repeat steps 2-3 until convergence
```

```
Objective: Minimize Σ Σ ||xi - μk||²
```

Core Assumptions

- Clusters are spherical and equal-sized
- Features should be scaled/normalized
- Number of clusters K must be specified

Pros

- Simple and fast (O(nKi) complexity)
- Scales well to large datasets
- Easy to implement and interpret
- Works well when clusters are roughly spherical

Cons

- Must specify K in advance
- Sensitive to initial centroid placement
- Sensitive to outliers
- Fails with non-spherical or unequal-size clusters

Finding Optimal K

- **Elbow Method** — plot inertia vs K, find the elbow point
- **Silhouette Score** — measures how similar points are to their own cluster vs others
- **Gap Statistic** — compares inertia to random baseline

When Not to Use

- Non-spherical or irregularly shaped clusters
- Clusters of very different sizes or densities
- Data with many outliers

Real-World Use Cases

- Customer segmentation
- Image compression
- Document clustering
- Anomaly detection

Interview Tips

> **Q: How does K-Means++ differ from standard K-Means?**
> K-Means++ uses a smarter initialization — centroids are chosen with probability proportional to their distance from existing centroids, reducing the chance of poor initialization.

---

10. Hierarchical Clustering

What Is It?

Hierarchical Clustering builds a tree of clusters (**dendrogram**) without requiring K to be specified in advance. It can be **agglomerative** (bottom-up, merge clusters) or **divisive** (top-down, split clusters). Agglomerative is most common.

Key Logic

```
Agglomerative (bottom-up):
1. Start: each point is its own cluster
2. Merge the two closest clusters
3. Repeat until one cluster remains
4. Cut dendrogram at desired level to get clusters

Linkage criteria:
- Single linkage: min distance between clusters
- Complete linkage: max distance between clusters
- Average linkage: average distance between clusters
- Ward linkage: minimize within-cluster variance
```

Core Assumptions

- A meaningful distance metric exists for your data
- No assumption about cluster shape (depends on linkage)

Pros

- No need to specify K in advance
- Produces intuitive dendrogram visualization
- Can reveal hierarchical structure in data
- Works with any distance metric

Cons

- Very memory and compute intensive $O(n^2 \log n)$
- Cannot undo merges (greedy algorithm)
- Sensitive to noise and outliers
- Does not scale to very large datasets

When Not to Use

- Very large datasets
- When computational resources are limited
- When you need fast clustering

Real-World Use Cases

- Gene expression analysis
- Social network analysis
- Document clustering
- Phylogenetic trees in biology

---

11. Principal Component Analysis (PCA)

What Is It?

PCA is a dimensionality reduction technique that transforms high-dimensional data into fewer dimensions while preserving maximum variance. It finds new axes (principal components) that capture the most information.

Key Logic

```
1. Standardize the data
2. Compute the covariance matrix
3. Compute eigenvectors and eigenvalues
4. Sort eigenvectors by eigenvalues (descending)
5. Select top K eigenvectors
6. Transform data to new K-dimensional space
```

Key Concepts

- **Principal Components** — new axes aligned with maximum variance
- **Explained Variance Ratio** — how much variance each component captures
- **Scree Plot** — visualizes explained variance vs components

Core Assumptions

- Large variance = important information
- Linear relationships between features

- Data should be scaled before applying PCA

Pros

- Reduces dimensionality and noise
- Speeds up downstream ML models
- Removes multicollinearity
- Good for visualization (project to 2D/3D)

Cons

- Principal components are hard to interpret
- Loses some information (always a tradeoff)
- Only captures linear relationships
- Sensitive to feature scaling

When Not to Use

- When all features are equally important
- When interpretability of features is required
- When non-linear relationships are important (use t-SNE or UMAP instead)

Real-World Use Cases

- Image compression
- Face recognition (Eigenfaces)
- Noise reduction
- Visualization of high-dimensional data

Interview Tips

> **Q: What is the difference between PCA and t-SNE?**
> PCA is linear and preserves global structure, great for dimensionality reduction before ML. t-SNE is non-linear and preserves local structure, great for visualization only.

---

12. Neural Networks (MLP)

What Is It?

A Multi-Layer Perceptron (MLP) is a fully connected feedforward neural network with an input layer, one or more hidden layers, and an output layer. Each neuron applies a weighted sum of inputs followed by a non-linear activation function.

Key Logic

```
Forward Pass:
z = W·x + b
a = activation(z)

Activation Functions:
- ReLU: max(0, x)            ← most common for hidden layers
- Sigmoid: 1/(1+e^-x)       ← binary classification output
- Softmax: e^xi/Σe^xj       ← multi-class output
- Tanh: (e^x - e^-x)/(e^x + e^-x)

Backpropagation:
Compute gradients using chain rule and update weights with gradient descent
```

Core Assumptions

- Enough data to learn complex patterns
- Features should be scaled/normalized
- Sufficient compute resources

Pros

- Can learn highly complex non-linear patterns
- Universal function approximator
- Works across many data types with the right architecture
- State of the art when data is abundant

Cons

- Needs large amounts of data
- Computationally expensive
- Many hyperparameters to tune

- Black box — hard to interpret
- Prone to overfitting on small datasets

When Not to Use

- Small datasets
- Limited compute resources
- When interpretability is required
- When simpler models achieve similar accuracy

Key Hyperparameters

- Number of layers and neurons per layer
- Learning rate and optimizer (Adam, SGD, RMSProp)
- Batch size and number of epochs
- Dropout rate for regularization
- Activation functions

Real-World Use Cases

- Image classification
- Natural language processing
- Speech recognition
- Tabular data (as alternative to tree-based models)

---

13. Convolutional Neural Network (CNN)

What Is It?

CNNs are specialized neural networks designed for **grid-like data** such as images. They use **convolutional layers** to automatically learn spatial hierarchies of features — from simple edges in early layers to complex objects in deeper layers.

Key Logic

```
Convolution: (f * g)(t) = Σ f(τ)g(t-τ)

Key layers:
- Convolutional layer: applies filters to detect features
- Activation (ReLU): introduces non-linearity
- Pooling layer: reduces spatial dimensions (Max/Avg pooling)
- Fully connected layer: final classification/regression
```

Core Assumptions

- Input has grid-like spatial structure (images, audio spectrograms)
- Local patterns are meaningful (edges, textures)
- Patterns are translation-invariant

Pros

- Automatically learns spatial features
- Parameter sharing reduces model size
- Translation invariant
- State of the art for image tasks

Cons

- Very high resource demand (GPU required)
- Needs large amounts of labeled data
- Long training time
- Less effective for sequence or tabular data

When Not to Use

- Sequence data (use RNN or Transformer)
- Tabular data (use tree-based methods)
- When labeled data is very limited

Famous Architectures

- **LeNet** — first CNN, handwritten digit recognition
- **AlexNet** — ImageNet breakthrough 2012

- **VGG** — very deep, simple architecture
- **ResNet** — skip connections, solved vanishing gradient
- **EfficientNet** — best accuracy/compute tradeoff

Real-World Use Cases

- Self-driving vision systems
- Medical image analysis
- Face recognition
- Object detection (YOLO, Faster R-CNN)

---

14. Recurrent Neural Network (RNN)

What Is It?

RNNs are neural networks with **loops** that allow information to persist across time steps. They process sequential data by maintaining a hidden state that captures context from previous inputs. LSTMs and GRUs are improved variants that solve the vanishing gradient problem.

Key Logic

```
Standard RNN:
ht = tanh(Wh·ht-1 + Wx·xt + b)
yt = Wy·ht + by

LSTM Gates:
- Forget gate: what to discard from cell state
- Input gate: what new info to store
- Output gate: what to output

GRU (simplified LSTM):
- Update gate: combine forget and input gates
- Reset gate: how much past info to forget
```

Core Assumptions

- Data has sequential or temporal structure
- Order of inputs matters
- Dependencies exist across time steps

Pros

- Designed for sequential data
- Shares parameters across time (efficient)
- Can model variable-length sequences
- LSTMs capture long-term dependencies well

Cons

- Vanishing gradient problem (standard RNN)
- Slow to train (sequential computation)
- Struggles with very long sequences
- Largely replaced by Transformers for NLP tasks

When Not to Use

- Very long sequences (use Transformer)
- When parallel computation is needed
- Non-sequential data

Real-World Use Cases

- Stock price prediction
- Speech recognition
- Time series forecasting
- Language modeling (largely superseded by Transformers)

---

15. Transformer (BERT, GPT)

What Is It?

Transformers revolutionized NLP and AI. They use **self-attention mechanisms** to process entire sequences in parallel, capturing relationships between all positions simultaneously. BERT is encoder-only (understanding). GPT is decoder-only (generation). T5/BART use encoder-decoder.

Key Logic

```
Self-Attention:
Attention(Q,K,V) = softmax(QK^T / √dk) · V

Where:
Q = Query matrix
K = Key matrix
V = Value matrix
dk = dimension of keys

Multi-Head Attention:
Run attention multiple times in parallel with different learned projections
```

Core Assumptions

- Large amounts of training data
- Sufficient compute (GPU/TPU clusters)
- Tokenized text input

Pros

- Captures long-range dependencies
- Highly parallelizable (fast training)
- Transfer learning — pre-train once, fine-tune for many tasks
- State of the art across NLP, vision, audio

Cons

- Enormous compute and memory requirements
- Large model size
- Expensive to train from scratch
- Not ideal for very small datasets or projects

When Not to Use

- Small projects with limited compute
- Simple structured data (use tree-based models)
- When interpretability is required

Famous Models

- **BERT** — bidirectional, great for understanding tasks
- **GPT-4** — autoregressive, great for text generation
- **T5** — text-to-text framework, versatile
- **Vision Transformer (ViT)** — transformers for images
- **Whisper** — transformers for speech

Real-World Use Cases

- ChatGPT, Claude, Gemini
- Machine translation
- Document summarization
- Code generation (GitHub Copilot)

---

16. Autoencoders

What Is It?

Autoencoders are unsupervised neural networks that learn to compress data into a lower-dimensional **latent representation** (encoder) and then reconstruct it (decoder). They are trained to minimize reconstruction error.

Key Logic

```
Encoder: x → z (compress to latent space)
Decoder: z → x̂ (reconstruct from latent space)

Loss: Reconstruction Loss = ||x - x̂||²

Variants:
- Denoising Autoencoder: add noise to input, reconstruct clean output
- Variational Autoencoder (VAE): learns probabilistic latent space
- Sparse Autoencoder: regularize latent space to be sparse
```

Core Assumptions

- Data has a lower-dimensional underlying structure
- Symmetric network architecture works best

Pros

- Unsupervised — no labels required
- Effective for denoising and compression
- Can generate new data (VAE)
- Useful for anomaly detection

Cons

- Can overfit and memorize rather than generalize
- Black box — latent space hard to interpret
- Reconstruction quality depends heavily on architecture
- Outperformed by GANs for generation tasks

When Not to Use

- When no compression benefit exists
- When labeled data is available (use supervised learning instead)
- When generation quality is critical (use GANs or Diffusion Models)

Real-World Use Cases

- Fraud detection (anomaly detection)
- Image denoising
- Data compression
- Drug discovery (molecular generation)

---

17. DBSCAN

What Is It?

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups together points that are closely packed in dense regions, while marking outliers in low-density regions as noise. Unlike K-Means, it does not require specifying K and can find arbitrarily shaped clusters.

Key Logic

```
Key Concepts:
- Core point: has at least minPts neighbors within radius ε
- Border point: within ε of a core point but fewer than minPts neighbors
- Noise point: not a core or border point

Algorithm:
1. For each point, find all points within distance ε
2. If core point, start a new cluster
3. Expand cluster by adding density-reachable points
4. Points not assigned to any cluster = noise/outliers
```

Core Assumptions

- Clusters are defined by density, not shape
- A meaningful distance metric exists
- Density is relatively uniform across clusters

Pros

- Does not require specifying number of clusters
- Finds arbitrarily shaped clusters
- Robust to outliers (marks them as noise)
- No assumption of spherical clusters

Cons

- Fails when clusters have very different densities
- Sensitive to ε and minPts parameters
- Struggles with high-dimensional data
- Memory intensive for large datasets

When Not to Use

- Varying density clusters
- Very high-dimensional data
- When cluster count must be specified

Real-World Use Cases

- Geospatial clustering (crime hotspots, delivery zones)
- Anomaly detection in network traffic
- Image segmentation
- Social media event detection

Quick Reference Cheatsheet

Algorithm	Type	Best For	Avoid When	Example
Linear Regression	Supervised	Continuous prediction	Strong non-linearity	House prices
Logistic Regression	Supervised	Binary classification	Highly non-linear data	Spam detection
Decision Tree	Supervised	Classification & regression	Noisy complex data	Loan default
Random Forest	Supervised	High accuracy ensemble	Real-time needed	Fraud detection
Gradient Boosting	Supervised	Best tabular accuracy	Interpretability needed	Credit scoring
SVM	Supervised	High-dim classification	Large noisy datasets	Face recognition
KNN	Supervised	Few-shot classification	High-dimensional data	Recommender
Naive Bayes	Supervised	Text classification	Correlated features	Sentiment analysis
K-Means	Unsupervised	Customer segmentation	Non-spherical clusters	Segmentation
Hierarchical Clustering	Unsupervised	Data structure exploration	Very large datasets	Gene analysis
PCA	Dimensionality Reduction	Feature reduction	All features equally important	Image compression

Algorithm	Type	Best For	Avoid When	Example
Neural Networks (MLP)	Supervised	Complex pattern modeling	Small data, low compute	Image classification
CNN	Supervised	Image/video/spatial data	Sequence or text data	Self-driving vision
RNN	Supervised	Sequence modeling	Very long sequences	Stock prediction
Transformer	Supervised/Self-supervised	NLP, chat, translation	Small projects	ChatGPT
Autoencoders	Unsupervised	Compression, anomaly detection	No compression needed	Fraud detection
DBSCAN	Unsupervised	Arbitrary shape clustering	Varying density	Geo-spatial

---

Choosing the Right Algorithm

```
Is your data labeled?
├── YES → Supervised Learning
│   ├── Predicting a number? → Linear Regression, Neural Networks
│   └── Predicting a category?
│       ├── Simple/interpretable needed? → Logistic Regression, Decision Tree
│       ├── High accuracy needed? → Random Forest, Gradient Boosting
│       ├── Image/video data? → CNN
│       ├── Text/sequence data? → Transformer, RNN
│       └── High-dimensional? → SVM, Naive Bayes
└── NO → Unsupervised Learning
    ├── Finding groups/clusters?
    │   ├── Know number of clusters? → K-Means
    │   ├── Unknown cluster count? → DBSCAN, Hierarchical
    │   └── Arbitrary shapes? → DBSCAN
    └── Reducing dimensions?
        ├── Linear relationships? → PCA
        ├── Visualization only? → t-SNE, UMAP
        └── Anomaly detection? → Autoencoders
```

Made with ❤️ by @datasciencebrain | Follow for daily AI & Data Science content